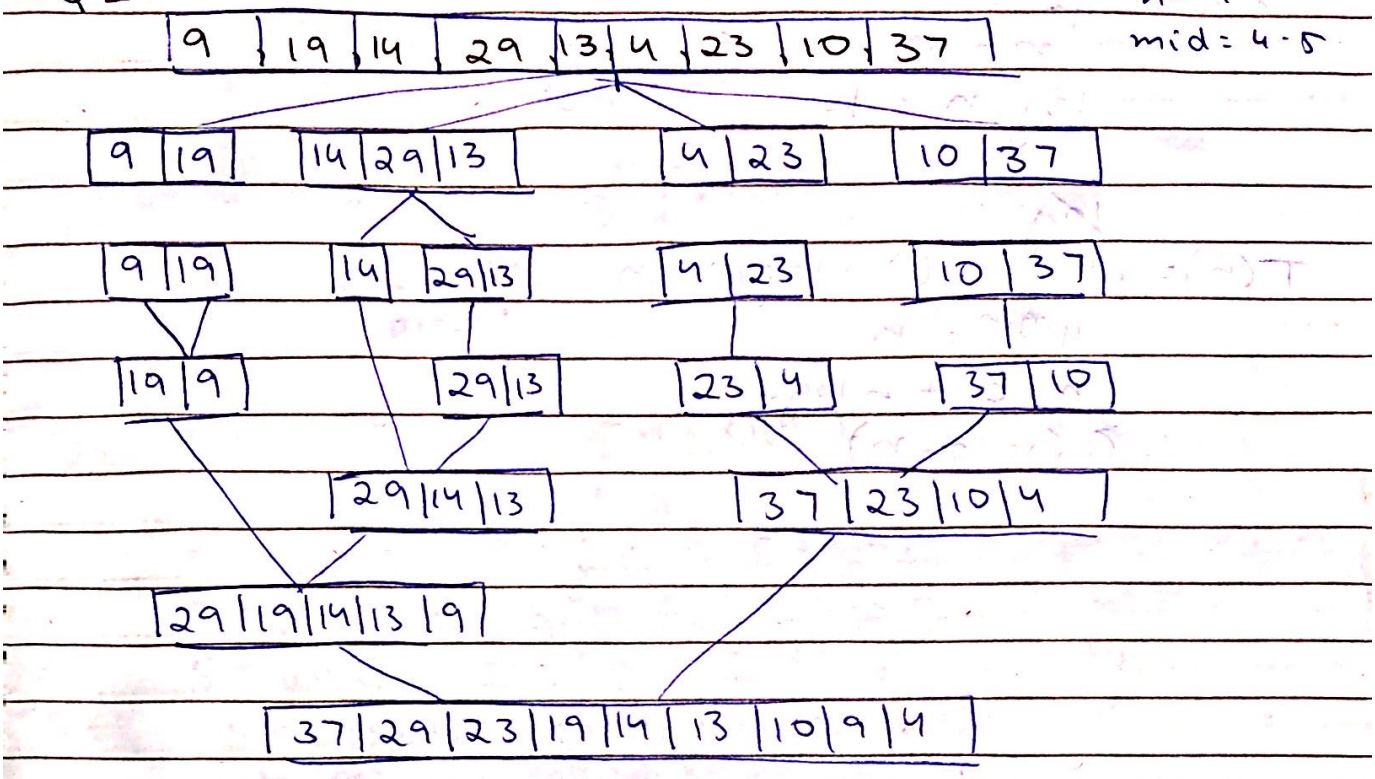
Laiba Fatima
22K-5195
BSE-5B

PAGE
DATE

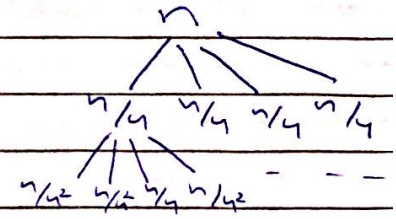
DAA Assignment 1

Q 1.



Time Complexity

$T(n) = 4T(n/4) + n$



$n/4 \Rightarrow T(n/4) = 4T(n/16) + n/4$

$T(n) = 4^2 T(n/4^2) + 2n$

$n/16 \Rightarrow T(n/16) = 4T(n/4^3) + n/4^3$

$T(n) = 4^3 T(n/4^3) + 3n$

after k steps

$T(n) = 4^k T(n/4^k) + kn$

PAGE
DATE

Q1

$\frac{n}{4^k} = 1 \leftarrow T(1)$

$n = 4^k$

$\log_4 n = k$

$T(n) = 4^{\log_4 n} T(1) + \log_4 n \cdot n$

$\therefore 4^{\log_4 n} = n$

$n \cdot c + n \log_4 n$

$O(n \log n)$

PAGE
DATE

Q2. $3n^3 + 5n^2 + 25n = \Omega(n^3)$

$f(n) \geq c * g(n)$ $\quad g(n) = n^3$

$3n^3 + 5n^2 + 25n \geq c * n^3$ $\quad c = 1$

$3n^3 + 5n^2 + 25n \geq 1 * n^3$ $\quad n_0 = ?$

$n=1$ $\quad 3(1)^3 + 5(1)^2 + 25(1) \geq 1 * (1)^3$ $\quad n_0 = 1$

$33 \geq 1$ $\quad n \geq n_0$

∴ proved $\quad \Omega(n^3)$ $c = 1$, $n \geq 1$

Q3. $5 \log_2 \log_2 n + 4 \log^2 n = O(?)$

$f(n) \leq c * g(n)$

$5 \log_2 \log_2 n + 4 \log_2^2 n \leq c * g(n)$ $\quad g(n) = \log_2^2 n$

$5 \log_2 \log_2 n + 4 \log_2^2 n \leq c * \log_2^2 n$ $\quad c = 5$

$5 \log_2 \log_2 n + 4 \log_2^2 n \leq 5 * \log_2^2 n$ $\quad n_0 = ?$

$n=1$ $\quad 5 \log_2 \log_2 (1) + 4 \log_2^2 (1) \leq 5 * \log_2^2 (1)$ $\quad * n \geq n_0$

$\log_2(0) \rightarrow$ undefined ×

$n=2$ $\quad 5 \log_2 \log_2 (2) + 4 \log_2^2 (2) \leq 5 * \log_2^2 (2)$ $\quad n_0 = 2$

$0 + 4 \leq 5$

$4 \leq 5$

hence, ~~∴ proved~~

$O(\log_2^2 n)$, $c = 5$, $n \geq 2$

Q4. i. outer loop (i)

for (i = n/2 ; i > 1 ; i /= 6) $\quad$ terminates $i \leq 1$

$-$ assume $i = \frac{n}{2 \times 6^k}$

$\frac{n}{2 \times 6^k} \leq 2 \times 6^k$

$\frac{n}{2} \leq 6^k$

$k \geq \frac{n}{2} : 6^k$

i
$n/2$
$n/2 \times 6$
$n/2 \times 6^2$
$n/2 \times 6^3$
⋮
$n/2 \times 6^k$

$k = \log_6 (n/2)$

$O(\log_c n)$

PAGE
DATE

Qu i)

mid loop (j)

for (j = 2; j <= n; j *= 4)

j
2
2×4
2×4²
⋮
2×4^K = 2×2^{2K} = 2^{1+2K}

terminates when $j > n$

$2^{1+2K} > n$

$2^{1+2K} = n$

$1 + 2K = \log_2(n)$

∴ ignore constants so

$O(\log_2 n)$

(also shown as $\log_4 n$ if we take 4^K)

inner loop (k)

for (k = 0; k <= j, k *= 3)

terminates when $k > j$

also depends on the j^{th} loop

k
0
0*3
0*3
⋮

runs infinitely

But overall the time complexity (if we ignore the infinite k loop and assume k is starting from 1, it would make k loop complexity as $O(\log_3 n)$ so the time complexity for entire ~~prog~~ algorithm

$i \times j \times k$

$\log n \times \log n \times \log n$

$O(\log^3 n)$

PAGE
DATE

Q4. ii) Outer loop (i)

for (i = n ; i > 1 ; i = i/2)

i
n
n/2
n/4
⋮
$n/2^k$

terminates $i \leq 1$

$n/2^k \leq 1$

$n \leq 2^k$

$k = \log_2 n$

$O(\log_2 n)$

Inner loop (j)

for (j = 2 ; j <= n ; j += 1)

j
2
3
4
5
⋮
k

terminates when $j > n$

$k > n$

$k = n$

$O(n)$

Overall time complexity

$i \times j$

$\log_2 n \times n$

$O(n \log_2 n)$

PAGE
DATE

Q5.

$\log\log n^2 \quad \log^2 n \quad n^{-2} \quad \log(2^{\log(n^2)}) \quad n^{1/2} \quad n^2$

Simplify for easier calculation

~~Apply log~~

$\log(2\log n)$	$(\log n)^2$	$\frac{1}{n^2}$	$\log(2^{2\log n})$	$n^{1/2}$	n^2
$\log 2 + \log\log n$	$\log n \log n$		$\log(2^{\log n})^2$	$\sqrt{n}$	
$1 + \log\log n$			$\log(n^{\log 2})^2$	$n^{1/2}$	
~~$\log\log n$~~			$\log(n^2)^2$		
			$2\log n$		

Mathematically

n	$1+\log\log n$	$\log n \log n$	$\frac{1}{n^2}$	$2\log n$	$n^{1/2}$	n^2
2^1	$1+0=1$	1	$1/4$	$2(1)=2$	$\sqrt{2}$	4
2^4	$1+2=3$	16	$1/256$	$2(4)=8$	$=8$	256
2^8	$1+3=4$	64	$1/65536$	$2(8)=16$	$=16$	65536
2^{128}	$1+7=8$	16384	$\frac{1}{2^{128}}$	$2(128)=256$	1.84×10^{19}	$(2^{128})^2$
	(2)	(4)	(1)	(3)	(5)	(6)

We confirm values on larger n values so non-decreasing / increasing order is:

$$n^{-2} < \log\log n^2 < \log(2^{\log(n^2)}) < \log^2 n < n^{1/2} < n^2$$

Q6. We measure efficiency of an algorithm by its speed. we measure speed in terms of the operations performed and the no. of inputs.

We use Asymptotic Notation to determine the speed or time complexity : Big O, Big Ω, Big Θ

~~When f(n)~~ Big O is ~~if~~ $f(n) \in O(g(n))$ if there are positive constants c & $n_0 > 0$ such that $f(n) \leq c * g(n)$ for all $n \geq n_0$.

PAGE
DATE

Q6. Big Ω is $f(n) \in \Omega(g(n))$ if there exists positive constants c & n_0 such that $f(n) \geq c * g(n)$ for all $n \geq n_0$.

Big Θ is $f(n) \in \Theta(g(n))$ if there exists positive constants c_1, c_2, & n_0 such that

$c_1 \times g(n) \leq f(n) \leq c_2 * g(n)$ for all $n \geq n_0$

- Algorithms : we will use integer functions & inputs
- n is real variable, n is integer variable

But the definitions of $O(n)$, $\Omega(n)$ & $\Theta(n)$ is same for n

Q7. i) $n = w\sqrt{n}$

$f(n) > c \cdot g(n)$

$n > \frac{1}{} \cdot \sqrt{n}$

$n > \sqrt{n}$

$n = 1$ $\quad 1 > \sqrt{1}$ $\quad \times$

$n = 2$ $\quad 2^1 > \sqrt{2}$ $\quad \checkmark$

$g(n) = \sqrt{n}$

$c = 1$

$n_0 = ?$

$n \geq 2$

$n = w\sqrt{n}$ is True for $c = 1$ & $n \geq 2$

ii) $f(n) = O(g(n))$ then $g(n) = \Omega(f(n))$

Assume $f(n) = n$ $\quad g(n) = n^2$

$f(n) = O(g(n))$

$n = O(n^2)$

$f(n) \leq c * g(n)$

$c = 1$ $\quad n \leq \frac{1}{} * n^2$

$n \leq n^2 \rightarrow O(n^2)$

$n_0 = ?$ $\quad 1 \leq 1$

$n \geq 1$

$g(n) = \Omega(f(n))$

$n^2 \quad c = 1 = \Omega(n)$

$g(n) \geq c * f(n)$

$n^2 \geq \frac{1}{} * n \quad c = 1$

$n^2 \geq n \rightarrow \Omega(n)$

$1 \geq 1 \quad n \geq 1$

So True

PAGE
DATE

Q7

iii) $f(n) = O(f(n)^2)$

Assume $f(n) = \frac{1}{n}$ so presumes $O((\frac{1}{n})^2)$

$f(n) \leq c * g(n)$

$\frac{1}{n} \leq c \cdot \frac{1}{n^2}$

$\frac{1}{n} \leq c \cdot \frac{1}{n \times n}$

$1 \leq c \cdot \frac{1}{n}$

∴ constant is not less than a decreasing function

so False

iv) if $O(f(n))$ & $\Omega(f(n))$ then $\Theta(f(n))$

Assume $f(n) = 2n+3$ $g(n) = n$

$O(f(n))$

$f(n) \leq c \cdot g(n)$

$2n+3 \leq \underline{5}\, n$ $c=5$

$2(1)+3 \leq 5(1)$ $n_0 = ?$, $n=1$

$5 \leq 5$

$O(n)$

$\Omega(f(n))$

$f(n) \geq c \cdot g(n)$

$2n+3 \geq c \cdot n$ $c=1$

$2n+3 \geq \underline{1} \cdot n$ $n_0 = 1$

$2(1)+3 \geq 1(1)$ $n=1$

$5 \geq 1$

$\Omega(n)$

$\Theta(f(n))$

$c_1 \cdot g(n) \leq f(n) \leq c_2 \cdot g(n)$

$c_1 \cdot n \leq 2n+3 \leq c_2 \cdot n$

$1 \cdot n \leq 2n+3 \leq 5n$

$n \leq 2n+3 \leq 5n$

$n_0 = 1$ $1 \leq 5 \leq 5$ → $\Theta(n)$

so True

PAGE
DATE

Q8. Polynomial Time

1. Input n no. of process — O(1)
2. Input time for each process — O(1), burst_time[i]
3. For every p[i] ; time_left [i] = burst_time [i] - O(1)
4. for (i = 0 to n-1) { — } O(n²)
   for (j = 0 to n-i-1) {
   if burst_time [j] > burst_time [j+1] {
   swap [burst_time [j] , burst_time [j+1]
   swap (process [j] , p [j+1]);
   } } }
5. Initialize sum = 0, turnaroundtime = 0 — O(1)
6. for (i = 0 to n) { — O(n)
   sum = sum + burst_time [i]
   turnaroundtime = turnaroundtime + sum
   }
7. Return total turnaroundtime → O(n²)

Logarithmic Time

* Input process [n]
* Function Mergesort (Process)
  mid = Process.length /2 ;
  left [] = Process [0 -> mid]
  right [] = Process [mid -> n]
  merge (left)
  merge (right)

function merge (left, right) {
  initialize i = 0 , j = 0, k = 0
  while (i < left.length && j < right.length)
  if (left [i] < right [j])
  Process [k++] = left [i++]

PAGE
DATE

Q8. else

Process [k++] = right [j++]

while i < left.length

Process [k++] = left [i++]

while j < right.length

Process [k++] = right [j++]

Sum = 0, turnaround time : 0

For each burst time in Processes

Sum += burst_time

turnaroundtime += sum

Return turnaround time

O(nlogn)